Back to Basics

Creating Shopware Plugins from Scratch — Knowledge Check

Shizuka's Product Badge story · 15 min · 26 pts · Pass at 17 pts

Name: _____ Date: _____ Score: ____ / 26

Tick ☐, circle True/False, write on lines. Bonus is extra credit.

Part A — What a plugin is (6 pts)

1. Doraemon says: "A Shopware plugin is a Symfony bundle, plus extras." Which extras? (tick **two**) **2 pts**

- ☐ **A** Lifecycle hooks (`install`, `activate`, `update`, `uninstall`), migrations, plugin config
- ☐ **B** DI container, event subscribers and routes — these come from Symfony already
- ☐ **C** A second database server built into every plugin
- ☐ **D** A standalone Shopware that runs without Symfony

2. In Shizuka's `composer.json`, what makes the package a **plugin**? (tick **two**) **2 pts**

- ☐ **A** `"type": "shopware-platform-plugin"`
- ☐ **B** `"type": "library"` — that is enough
- ☐ **C** `extra.shopware-plugin-class` pointing at the class extending `Plugin`
- ☐ **D** `extra.shopware-app-name`

3. What is `extra.label` for, and which locale is the fallback? **2 pts**

_____

_____

Part B — Lifecycle + where plugins live (8 pts)

4. Match the hook to the job (write letter): `install()` ____ `activate()/deactivate()` ____ `update()` ____ `uninstall()` ____ **2 pts**

A. migrate data (schema lives in `Migration/`) · B. seed defaults, e.g. badge types · C. drop tables unless `keepUserData` · D. toggle what must not run while inactive

5. In `install()` Doraemon warns: `$this->container->get(MyService::class)` can fail. Why, and what should you use instead? **2 pts**

_____

_____

6. `custom/plugins/` by hand (ZIP copy) is the recommended way to install Store extensions. ☐ True ☐ False **1 pt**

7. `custom/static-plugins/` holds project extensions like ProductBadge, committed to the project repo and required via Composer path repository. ☐ True ☐ False **1 pt**

8. Shizuka asks where ProductBadge goes. Tick the **correct** row: **2 pts**

- ☐ **A** `custom/static-plugins/` — written for this project, versioned in Git
- ☐ **B** `custom/plugins/` — zipped upload is the modern default
- ☐ **C** `vendor/` by hand-editing files inside `vendor/`

Part C — Where the badge lives + storefront (5 pts)

9 . First wish: one fixed badge per product (Bestseller / Limited), set in admin, shown in listing + detail. Custom field or own entity? **2 pts**

- ☐ **A** Custom field (`CustomFieldSet` on product) — JSON column, admin select for free, no migration
- ☐ **B** Own entity — needed even for a single fixed label

10 . Name **two** limits of the custom-field approach that Shizuka hits with her **next** wish (own badges, several per product, filter by badge): **2 pts**

_____

_____

11 . Shopware picks up `custom-fields.xml` on `install()` and `update()` — no PHP or repository calls needed. ☐ True ☐ False **1 pt**

Part D — Own entity + storefront (7 pts)

12 . Shizuka's next wish: own badges, several per product, reusable across products. Why many-to-many? **2 pts**

- ☐ **A** One product carries several badges **and** one badge is reused across many products → mapping table `swag_product_badge_product`
- ☐ **B** One badge belongs to exactly one product, so a `product_id` column on the badge is enough
- ☐ **C** No relation is needed — badges are rendered from snippets only

13 . `Migration1791373708...`: which three things must be right? (tick **three**) **2 pts**

- ☐ **A** Three tables: `swag_product_badge`, `..._translation` (FK `language_id`), `..._product` (FK `product_id` + `product_version_id`)
- ☐ **B** All FKs `ON DELETE CASCADE`, `BINARY(16)` ids, `IF NOT EXISTS` + `getCreationTimestamp()`
- ☐ **C** `uninstall()` drops mapping → translation → main, and keeps data when `keepUserData()`
- ☐ **D** Badges stay in `customFields` JSON, so no tables are needed

14 . Doraemon overrides `component_product_badges_new`. Why wrap his badges in a **new** nested block? **3 pts**

> A: {% block component_product_badges_new %}{{ parent() }} ...badges... {% endblock %}
> B: {% block component_product_badges_new %}{{ parent() }}{% block component_product_badges_swag_product_badge %} ...badges... {% endblock %}{% endblock %}

Circle the better version and say in one line why:

_____

_____

Scoring

Band	Score	Meaning
🚀 Store-ready	22–26+	Could build ProductBadge again from scratch.
✅ Solid	17–21	Pass. Re-read one weak block.
🔧 In progress	< 17	Rewatch lifecycle + custom-field vs entity + Twig blocks.